

Introduction to DSA

Agenda

- C++ Review & STL
- Common Mistakes
- Introduction to Problem Solving
- Testing
- Brute Force Algorithm and Recursion

Agenda

- C++ Review
- Common Mistakes
- Introduction to Problem Solving
- Testing
- Brute Force Algorithm and Recursion

C++ Review: Basics

- Basic Data types
 - int
 - float
 - double
 - char
 - bool
 - void
- Pointers
 - `int * ptr = new int;`
 - `int * ptr = new int[10];`
 - `int * ptr = &x;`
 - `int x = *ptr;`

C++ Review: Basics

- If Conditions
- For and While Loop
- Arrays
 - `int arr1 [10];`
 - `int * arr2 = new int[N];`
- Structs
- Classes

C++ Review: Passing Variables to Functions

- **Passing by value** (Not recommended for large structures, check passing by constant reference)

- ```
int func(int x){
 x++;
 return x;
}
int x = 5;
cout << func(x); // printed 6, x is 5
```

- **Passing by reference**

- ```
int func(int &x){  
    x++;  
    return x;  
}  
int x = 5;  
cout << func(x); // printed 6, x is 6
```

C++ Review: Passing Variables to Functions

- Passing by pointer

- ```
int func(int * ptr){
 (*ptr)++;
 return *ptr;
}
int x = 5;
cout << func(&x); // printed 6, x is 6
```

- Passing by constant reference

- ```
int func(const int & x){  
    return x+1;  
}  
int x = 5;  
cout << func(x); // printed 6, x is 5
```

C++ Review: Reading from stdin

- How to read the following input from stdin into 3 integers?
- 1 5 7
- `int a, b, c;`
- `cin >> a >> b >> c;`

C++ Review: Namespace std

- Including (`using namespace std;`) in your code simplifies your code.
- Instead of having to write functions and variables like `std::cout` each time we directly write `cout`.
- However, using `namespace std;` causes **a lot of variables and functions** to be included in the current namespace which sometimes lead to ambiguity.
- It is a better practice to include the following code instead for each used std function, variable, or class.
 - `using std::cout;`
 - `using std::endl;`
 - `using std::vector;`

C++ Review: Standard Library

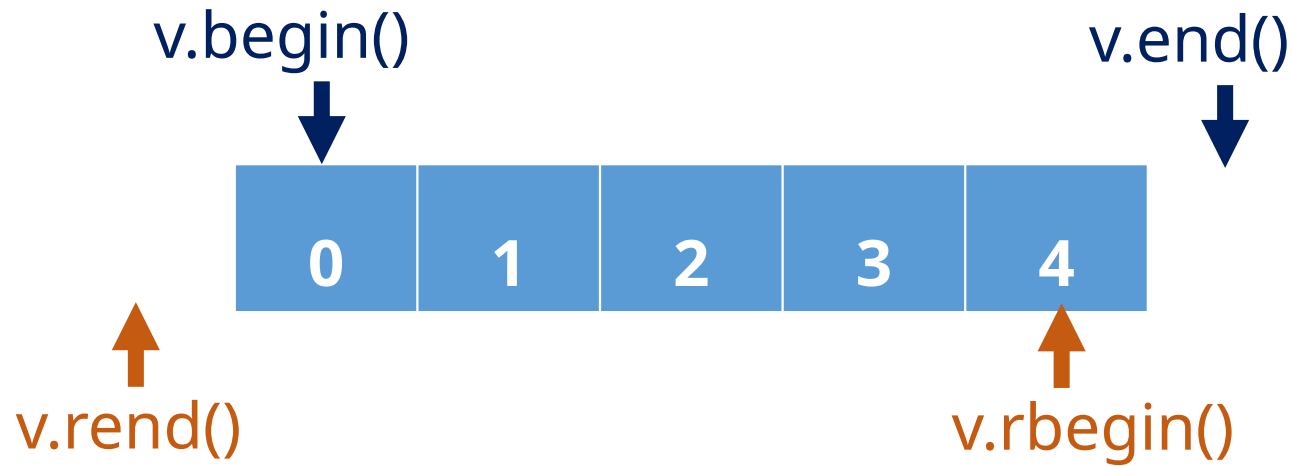
- C++ standard library offers
 - string classes
 - numeric classes
 - the standard version of stream I/O classes
 - basic memory allocation
 - exception classes
 - standard template library (STL)

C++ Review: Standard Template Library (STL)

- STL contains a lot of useful **containers** such as:
 - pair, vector, stack, queue, deque, set, map, bitset (an array of bits where each bit takes only 1 bit in memory), unordered_set (C++11), unordered_map (C++11)
- STL contains a lot of useful **algorithms** such as:
 - Sorting, Searching, Finding Maximum and Minimum
- STL contains **Iterators**

C++ Review: STL Iterators

- Iterators are objects like pointers that are used to access the elements of STL containers.
- To access the beginning of a vector:
 - `vector<int> vec(5);`
 - `vector<int>::iterator it = vec.begin();`
- To move to the next item, simply increment the iterator:
 - `it++;`
- Iterators support all operators that are supported by pointers such as `*` and `->`
- `end()` function returns a pointer that follows the last element. It holds no value and upon reaching it this means that one has finished iterating the container.
- To traverse a container in reverse order use `rbegin()` and `rend()`.



```
int arr[] = { 0,1,2,3,4 };
vector<int> v(arr, arr + 5);
// to print the vector in forward direction: 01234
for (auto it = v.begin(); it != v.end(); it++) //auto= vector<int>::iterator
    cout << *it;
// to print the vector in backward direction: 43210
for (auto it = v.rbegin(); it != v.rend(); it++) //auto=
vector<int>::reverse_iterator
    cout << *it;
```

C++ Review: Pairs

- `pair<int, char> p;`
 `p.first = 10;`
 `p.second = 'K';`

C++ Review: Vectors

- A vector is like a **dynamically allocated array**.
- Its **allocation, deallocation and resizing** are handled **automatically**.
- The needed includes:
 - **#include <vector>**
 - **using namespace std;**
- Creating a vector:
 - **vector<int> v;** **// creates a vector with 0 elements**
- Adding item to vector:
 - **v.push_back(10);** **// add a new element at the end (now v has 1 element)**
- Accessing vector element:
 - **cout << v[i];**

C++ Review: Vectors

- Iterating over vector (either using **indexes** or **iterators**):
 - `for(int i = 0; i < v.size(); i++)
 cout << v[i] << " ";`
 - `for(auto it = v.begin(); it != v.end(); it++)
 cout << *it << " ";`
- You can use your own struct/class in a vector.
 - `vector<mystruct> v;`

C++ Review: Vectors

- Initializing vector with 0 size:

- `vector<int> v2;` // creates a vector with 0 elements
- `v2.push_back(3);` // now the vector has 1 element

- Initializing vector with a size:

- `vector<int> v2 (N);` // creates a vector with N elements

- Initializing vector with a size and value:

- `vector<int> v2(N, val);` // creates a vector with N elements,
// each element has initial value: **val**

- Initializing vector with an array:

- `int arr[] = { 0,1,2,3,4 };`
- `vector<int> v (arr, arr + 5);` // **arr** is a pointer to the first element (like the **begin()** but for arr),
// **arr+5** is a pointer to the element after the last (like the **end()** but for arr)

```
vector<int> v (10, 3);  
v.push_back(5);
```

- How many elements are there in this vector?
- What is the correct way to update the value of last element to 100?

C++ Review: Set

- A set is a **sorted** container containing **unique** keys whenever an item is inserted it is placed in a sorted manner. → **Sorted + Unique**
- Sets are implemented as **BSTs**.
- They are very efficient for **search** operations. **$O(\log n)$**
- Example:
 - `set<int> myset;`
 - `myset.insert(2);`
 - `myset.insert(1);` // inserts elements sorted, now myset = 1 2
 - `set<int>::iterator it = myset.find(1);` // find: binary search $O(\log n)$
 - `if (it != myset.end())`
 `cout << "Found value";`
- To erase an element, use **`myset.erase(it)`** or **`myset.erase(value)`**.
- You can **iterate** on a set using **iterators** or **reverse iterators** (~~not indexes~~).

C++ Review: Map

- A **map** is like a **set** however it has a **key** and a **value**.

map<string, int> ages;

- Keys inserted **sorted and unique**.

- We use **keys as indexes**. Ex.: ages["ali"] = 20;

- Example:

- `map<int, int> mymap;`
- `mymap[100] = 0; // {key 100, value 0}`
- `mymap[100] = 5; // {100, 5}, the value will be updated`
- `int key = 1;`
- `auto it = mymap.find(key); // searches for key == 1`
- `if (it == mymap.end()) // if it is not found`
 - `mymap[key] = 0; // insert it in the map {key 1, value 0}`
 - `// now the map contains: {1,0} then {100,5} → sorted unique keys`
- `mymap[key]++; // increment the "value" of the element (key 1, value 1)`

Keys:

Values:

"Ahmed"	"ahmed"	"ali"	"ziad"
"	"		
30	20	20	10

- If you want the **key** of a set/map to be **your own struct**, you need to override the **< operator**.

C++ Review: Map

- Another Example:

- `map<string, int> mymap;`
- `mymap.insert({ "ziad", 10 });`
- `mymap.insert({ "ziad", 5 });` // will be found.
- `mymap.insert({ "Ahmed", 30 });`
- `mymap.insert({ "ahmed", 20 });`
- `mymap.insert({ "ali", 50 });`
- `for (auto it = mymap.begin(); it != mymap.end(); it++)`
`cout << it->first << " " << it->second << endl;`

Keys:

"Ahmed"

"ahmed"

"ali"

"ziad"

Values:

30

20

50

10

```
Ahmed 30
ahmed 20
ali 50
ziad 10
```

C++ Review: Other containers

- `Multiset` and `multimap` are like `set` and `map` but **allow duplicate keys**.
- `Unordered_set`, `unordered_map` are like `set` and `map` but are **not ordered**. They are larger in size but **faster in search**. $O(1)$. They are implemented as **hash tables**.
- In order to use a custom class/struct as a key to an `unordered_map` you need to provide your own **hash function**.

C++ Review: Sort and Binary Search

- `sort(startaddress, endaddress); // O(nlogn)`
Sorts a container that starts at `startaddress` and ends right before `endaddress`.
- Example
 - `sort( myvec.begin() , myvec.end() )`
 - `sort( myarr, myarr + arrSize )`
- `binary_search(startaddress, endaddress, value); // O(logn) but must be sorted`
Finds value in a sorted container using binary search.
- Check `lower_bound`, `upper_bound`, and `find` functions as well.

C++ Review: Trees Representation

- Representation of Node:

- `struct Node {`
 `int value;`
 `Node * left, * right; // or vector<Node*> children; if a general tree`
 `(>2 children)`
 `};`

- Representation of Binary Tree:

- `struct BinTree {`
 `Node* root;`
 `};`

C++ Review: Graphs Representation

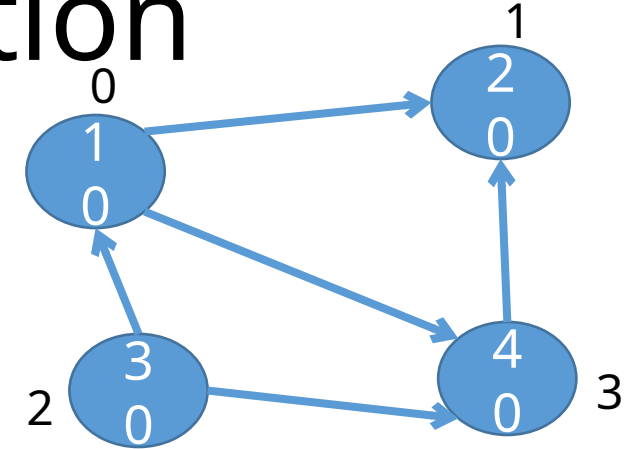
- Representation 1:

- ```
struct Node {
 int value;
 vector<Node*> adjacent;
};
```

- Representation 2 (Adjacency Matrix):

- ```
vector<int> values (N);  
vector<vector<int>> adjMatrix (N, vector<int> (N, 0));  
// n x n matrix with initial element value 0
```

- In an unweighted graph $\text{adjMatrix}[i][j] = 1$ if there exists an edge from i to j .
- In a weighted graph $\text{adjMatrix}[i][j] = W$ where W is the weight of the edge between i and j
- In an undirected graph $\text{adjMatrix}[j][i] = \text{adjMatrix}[i][j]$



Values:

10	20	30	40
----	----	----	----

To

	0	1	2	3
0	0	1	0	1
1	0	0	0	0
2	1	0	0	1
3	0	1	0	0

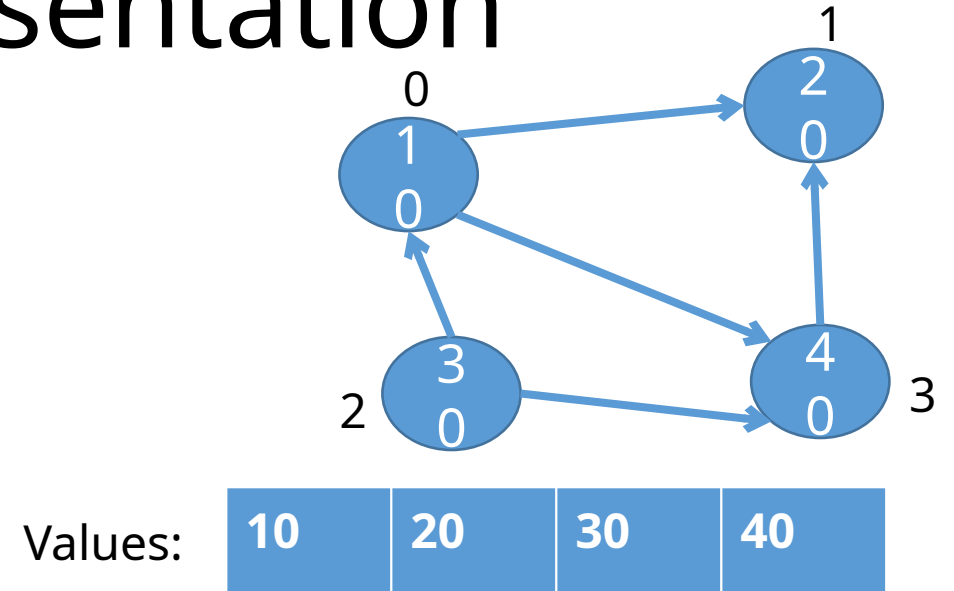
From

Adjacency Matrix

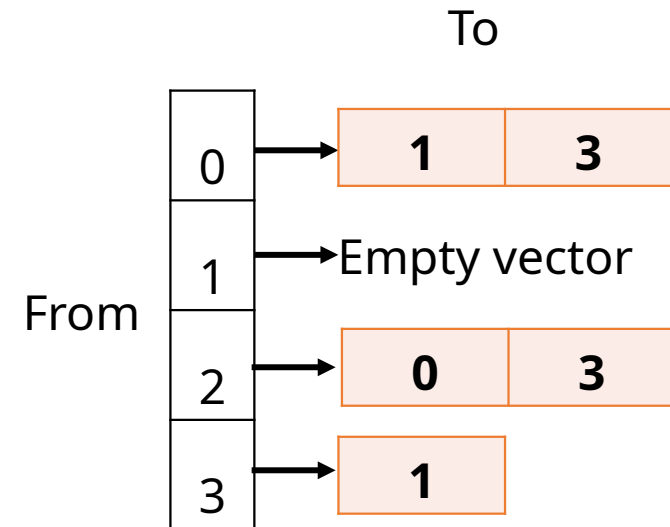
C++ Review: Graphs Representation

- Representation 3 (Adjacency List):

- `vector<int> values (N);`
`vector<vector<int>> adjList (N,`
`vector<int>());`
`// note that the 2nd dimension has empty vectors`



- We add `index j` to `adjList[i]` (using `push_back`) if there exists an edge from `i` to `j`.
`adjList[i].push_back(j);`
- We can make the list a `list of pairs` if we need to add `index and weight` for each edge.



C++ Review: Stack vs Heap Space

- All variables **allocated automatically** are stored in **stack**.
 - `int x;`
 - `int arr[10];`
 - `int* ptr = &x;`
- A **vector's headers** are stored in **stack** but **data** are allocated in **heap**.
 - `vector<int> v (100, 5);` // There are **100 elements in heap** but vector header is in stack
- When analyzing memory complexity sometimes we differentiate between **stack complexity** and **heap complexity**.

C++ Review: Stack vs Heap Space

- What happens when you call a function?
 - **Function parameters** are pushed to the **stack**.
 - The following function will cause a **stack overflow** although it does not allocate any memory because it does **not have an exit condition**.
 - ```
void f(){
 f();
}
```
- **N function calls** are equivalent to  **$O(N)$  stack space complexity**.
- Creating **an array of size N automatically** is equivalent to  **$O(N)$  stack space complexity**.
- Creating **an array of size N dynamically** is equivalent to  **$O(N)$  heap space complexity**.

# C++ Review: Other Remarks

- When using **vector** it is faster to:
  - Initialize **array size** on creation to avoid resizing
  - Use **indexing** (`arr[i]`) instead **push\_back** and **insert** functions
- When **passing vectors, maps, sets, strings, or any container** to a function, it is faster to pass them by **reference or constant by reference**. This **avoids copying of a large vector** with each call and can solve a lot **of timeout issues**.
- When you face a **segmentation fault** use a **debugger** to identify where and why it is occurring.

# Agenda

- C++ Review
- **Common Mistakes**
- Introduction to Problem Solving
- Testing
- Brute Force Algorithm and Recursion

# Common Mistakes: Integer Overflow

- `int` is 32 bits. Takes a value between -2147483648 and 2147483647
- `long long` is 64 bits. Takes a value between  $-(2^{63})$  to  $(2^{63})-1$
- Tip: use `long long` if your numbers are more than 9 digits ( $10^9$ ).

```
int a =
50000;
int b =
50000;
cout << a *
b;
```

-  
1794967296

```
int a = 50000;
int b = 50000;
long long c = a *
b;
cout << c;
-1794967296
```

```
int a = 50000;
int b = 50000;
long long c = (long long)a *
b;
cout << c;
2500000000
```

# Common Mistakes: Exceeding Time Limit

- Modern computers are quite fast and can process up to  $\approx 100M$  (or  $10^8$ ) operations in a **few seconds**.  
→ **avoid exceeding  $10^8$  complexity to stay within time bounds**
- **Tip: think for a faster algorithm if the algorithm complexity exceeds ( $10^8$ ).**

Example: Assume  
algorithm 1  
has  **$O(N^2)$  complexity**

- **If max  $N$  is  $10^3$ ,**  
then you can use  
algorithm 1  
→ because  $10^6$  is within  
time bounds

- **If max  $N$  is  $10^5$ ,**  
then you need to think of a  
faster algorithms,  
for example:  $O(N)$  or  $O(N \log N)$

# Common Mistakes: Undefined Behavior

- Incorrect Indexing of an array
  - `int a[10];`
  - `a[-1] = 0; a[10] = 0;`
- Using local variable without assignment
  - `int a;`
  - `a++;`
  - `cout << a;`
- Non void function without return
- When using a `map` calling `mymap[key]++` when `key` does not exist in the map causes undefined behavior.
- A lot of `undefined behavior` can be captured by `warnings`.



# Common Mistakes: Unnecessary Output

- Your **output format** should exactly match the problem specification.
- If the required output is one number you should only output one number.
- If you need to input data use cin only. Do not output **“please input x”**, this will cause you to get a wrong answer even if your answer is right.

# Agenda

- C++ Review
- Common Mistakes
- Introduction to Problem Solving
- Testing
- Brute Force Algorithm and Recursion

# Algorithmic Problems

- Precisely formulated problems
- Specific input and output format
- Constrained time and memory
- Requires knowledge of a specific algorithm

# Problem Solution

- Program steps
  - Read data in the specified format
  - Run your chosen algorithm
  - Output data in the specified format
- Code is then run by the judge against multiple test cases

# Judge Verdict

- Compilation Error (CE)
- Runtime Error (RTE)
- Time Limit Exceeded (TLE)
- Memory Limit Exceeded (MLE)
- Wrong Answer (WA)
- Accepted (AC)

# Test Cases

- Test cases are strictly formatted as specified in the problem.
- The range of each parameter is specified in the problem.
  - No need to make validation of the entered parameters that they are in range
- Do not make any assumptions that are not given in the problem.
  - For example, if the input array of a test case is: 10 30 50 90
  - Don't assume the array is entered sorted, unless this is stated explicitly in the problem description.

# Problem Example

## Find Maximum Number

You are given a list of numbers. Find and print the **maximum** of these numbers.

### Input Format

One line containing N which is the length of the list

One line containing N numbers separated by spaces

N is between 1 and  $10^8$ . Each number is between -10000 and 10000.

### Output Format

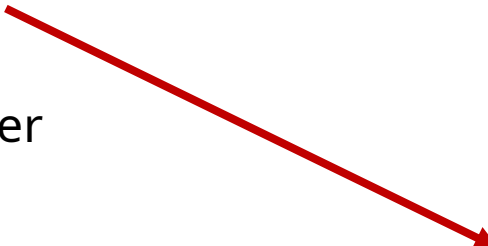
One line containing the maximum number

### Input Sample

5  
200 250 3000 -1 230

### Output Sample

3000



Note:

No need to check if the N and the numbers are in the specified range.  
It is guaranteed to be entered that way.

# Agenda

- C++ Review
- Common Mistakes
- Introduction to Problem Solving
- **Testing**
- Brute Force Algorithm and Recursion



# Testing: Why test your code?

- **Test correctness.**
  - Make sure your code outputs the **correct answer**.
- **Test reliability.**
  - Make sure your code does **not crash** in some test cases.
- **Test limits.**
  - Get an indication whether or not your test will pass **time and memory limits**.

# Testing: Tests Types

- **Samples tests**

- Sample tests are **the tests given in the problem**. Make sure that your code passes them.

- **Minimal test**

- Test the **minimal case of the input**. If the input is a list with size  $N$  and  $N$  varies between 1 and  $10^6$ , the minimal test case will be one number only with the minimum value possible.

- **Maximal test**

- It is the opposite of minimal test. Test your code against **the maximum** size input. You will need to generate the test case using another code.
- Here we do not test for correctness we test for limits and reliability.

- **Custom tests**

- Manually create tests that covers **different scenarios and corner cases** of the problem and **verify correctness** by either solving by hand or by a simple brute-force code.

# Testing: Stress Testing

- **Stress testing** is done automatically by running your code against multiple random tests.
- Stress testing loop consists of:
  - Generate a **random input**
  - **Run** your solution
  - Check if the output is correct
  - If the output is **wrong** or there is a **crash**, output the test case and stop
- To check **correctness** of the output, you can use a **simple brute force solution** to compare the results. However, this means the test cases have to be small.
- If you are only interested in finding **crashes**, no need to check for correctness.

# Agenda

- C++ Review
- Common Mistakes
- Introduction to Problem Solving
- Testing
- Brute Force Algorithm and Recursion

# Recursion

- A recursive function is a function that makes one or more recursive calls to itself. → this is called the general case.
- It should have a base case (exit condition) otherwise it will recurse forever.
- A good way to track/design your recursive function is drawing a recursion tree and tracing its base case and number of function calls.
- You should also exercise writing recursive relations.

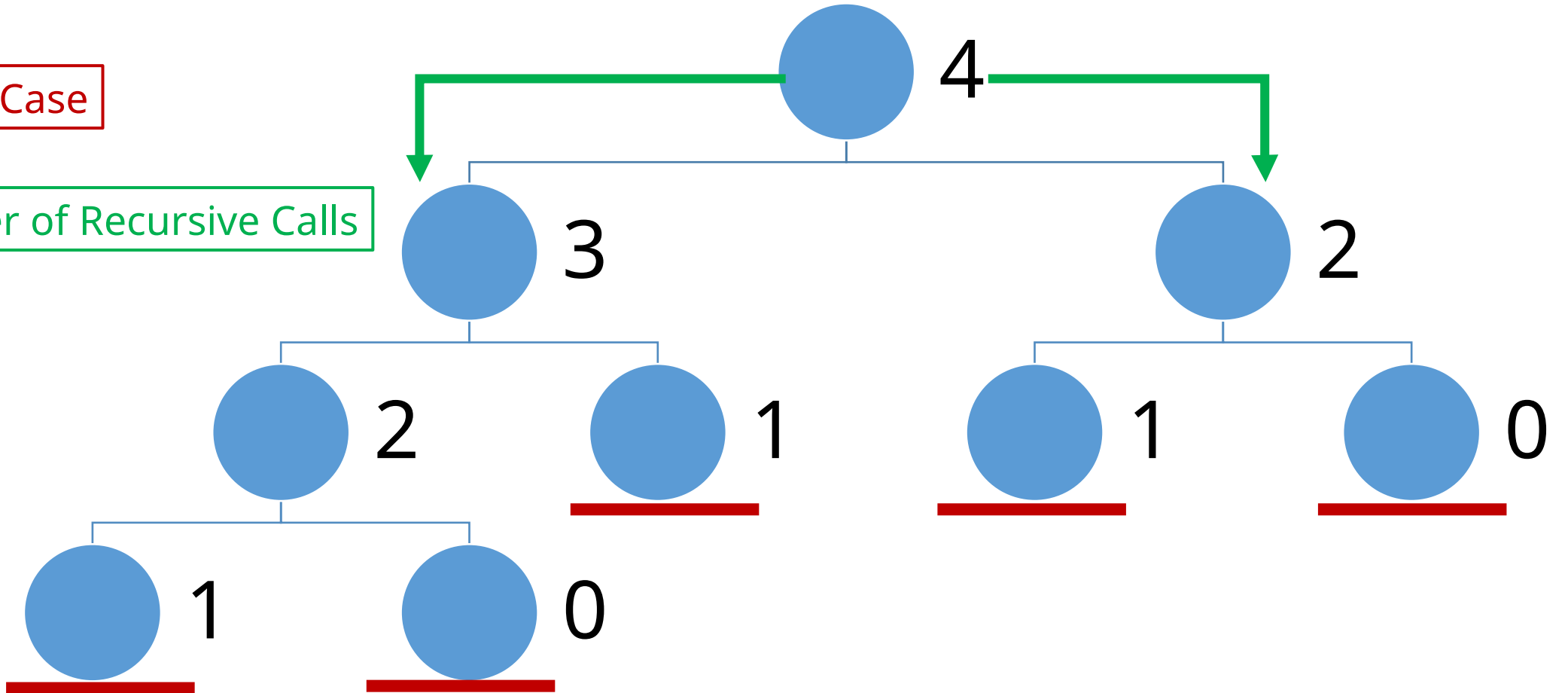
# Recursion

- If you wish to calculate Fibonacci number  $F(N)$  using recursion.
- $F(0) = 0$ ,  $F(1) = 1$ , and  $F(N) = F(N-1) + F(N-2)$
- Let's look at an example **recursive tree** of  $F(4)$

# Recursion

1- The Base Case

2- The number of Recursive Calls



# Recursion

- **From the recursion tree**, we can know two types of information:
  1. **The base case**: the last level in the tree, no more recursive calls
  2. **The number of recursive calls**: the number of branches in each node
- In case of **Fibonacci**:
  - **The base case** is  $F(0)$  and  $F(1)$ .
  - **The number of recursive calls** at each node is 2.
- ```
int F(int N){  
    if(N == 0)  
        return 0;  
    if(N == 1)  
        return 1;  
    return F(N-1) + F(N-2);  
}
```


Recursion:

Different ways to return results

- Getting the summation of all elements in an array recursively.

- Assume adding: 4 3 2 5

- Summing on the way down and return

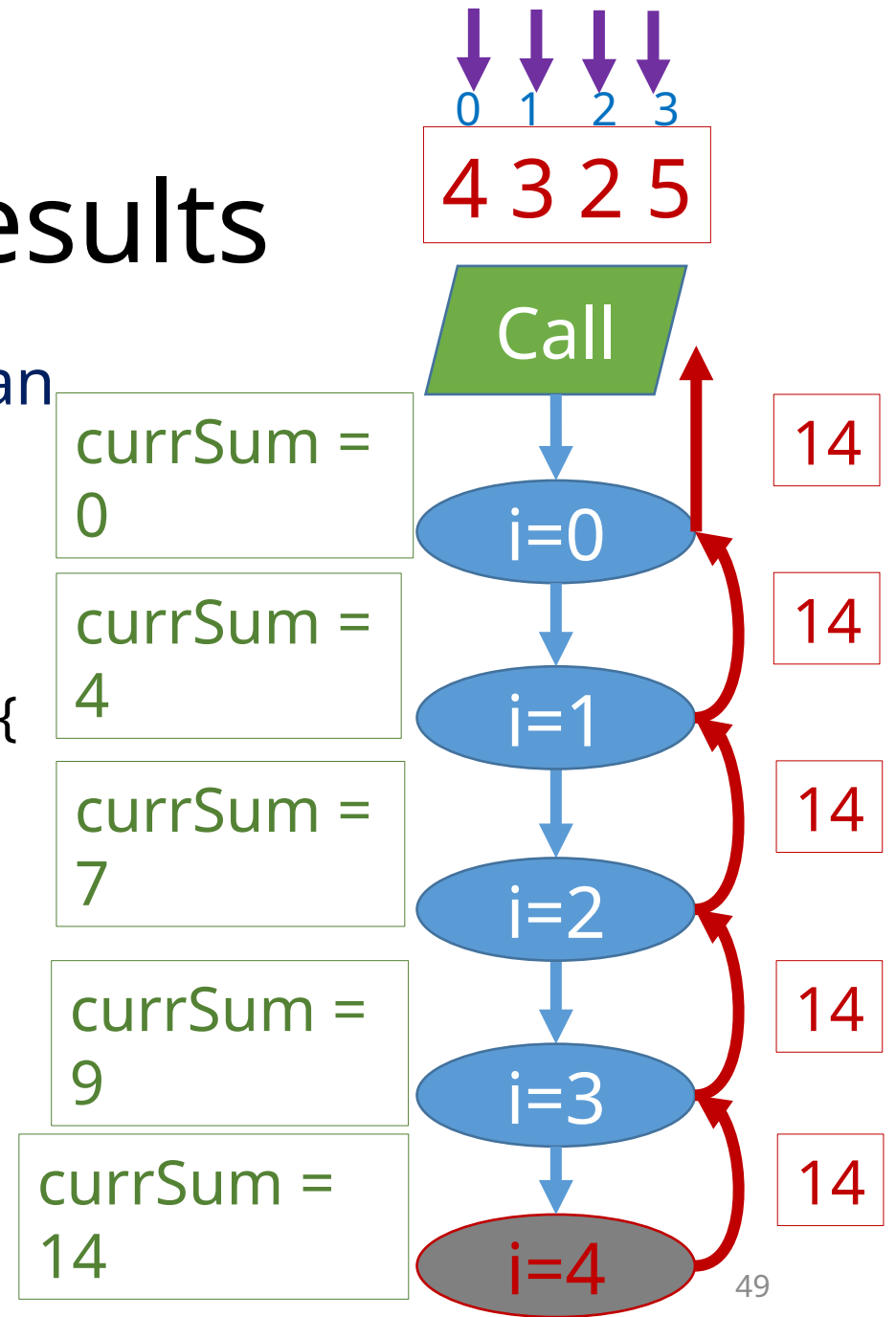
```
int arraySum(vector<int> arr, int currSum, int i){  
    if(i == arr.size())  
        return currSum;  
    currSum += arr[i];  
    return arraySum(arr, currSum, i+1);  
}
```

- Call:

```
int result = arraySum(vec, 0, 0);  
cout << result;
```

State

State Transition



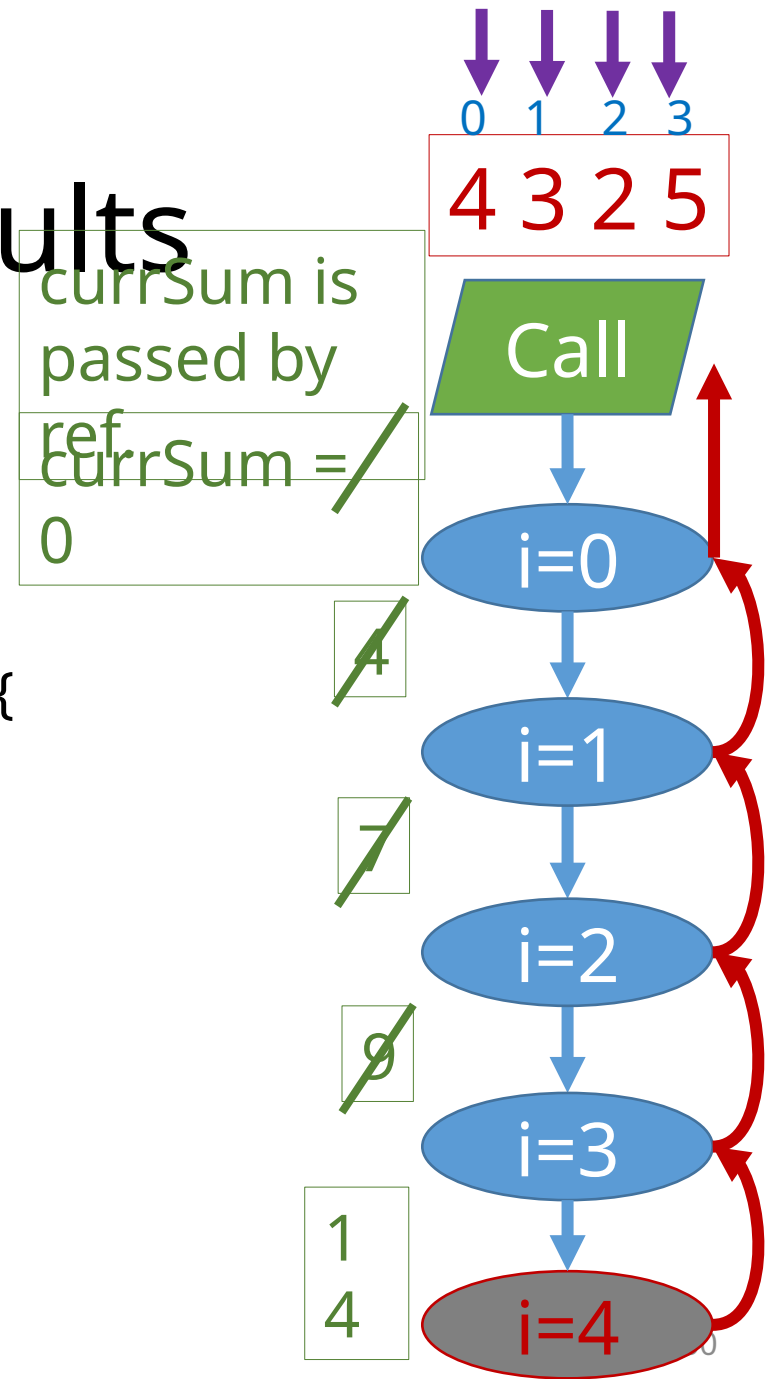
Recursion:

Different ways to return results

- Getting the summation of all elements in an array recursively.
- Summing on the way down and in a passed by reference parameter

```
void arraySum(vector<int> arr, int& currSum, int i){  
    if(i == arr.size())  
        return;  
    currSum += arr[i];  
    arraySum(arr, currSum, i+1);  
}
```

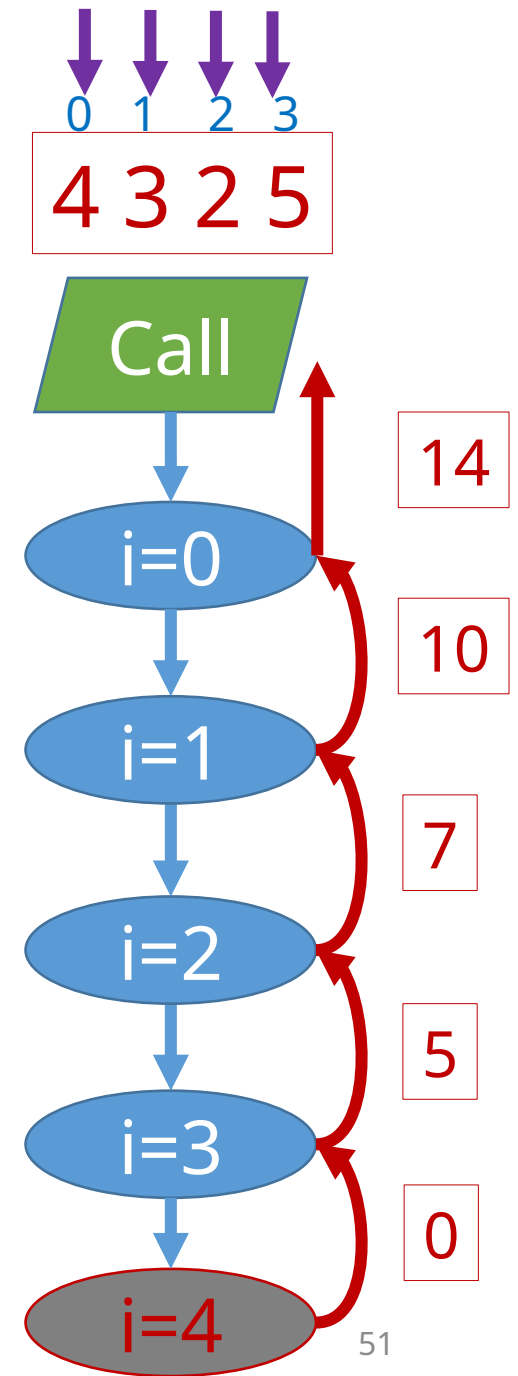
- Call:
 - `int currSum = 0;`
 - `arraySum(vec, currSum, 0);`
 - `cout << currSum;`



Recursion:

Different ways to return results

- Getting the summation of all elements in an array recursively.
- **Summing on the way up**
 - `int arraySum(vector<int> arr, int i){`
 `if(i == arr.size())`
 `return 0;`
 `return arraySum(arr, i+1) + arr[i];`
}
- Call:
 - `int result = arraySum(vec, 0);`
 - `cout << result;`



Brute Force Solutions

- Brute Force technique **enumerate the entire search space** and finds the solution.
- It is always correct for combinatorial problems yet **very slow**.
- It can be used to **verify faster solutions**.
- The **search space** is the set containing **all candidate solutions**. Let the search space be set **A**.

Search Space Example 1

Find Maximum Number

You are given a list of numbers. Find and print the **maximum** of these numbers.

Input Format

One line containing N which is the length of the list

One line containing N numbers separated by spaces

Output Format

One line containing the maximum number

Input Sample

5
200 250 3000 -1 230

Output Sample

3000

- Set **A** is {200, 250, **3000**, -1, 230}
- How to enumerate over search space?
 - Using a for loop on the input list.
`for (int i = 0; i < N; ++i)`

Search Space Example 2

Find Maximum Sum

You are given a list of numbers. Find **the maximum sum** that can be achieved by **summing of any unique two numbers**.

Input Format

One line containing N which is the length of the list

One line containing N numbers separated by spaces

Output Format

One line containing the maximum number

Input Sample

3

250 3000 -1

Output Sample

3250

- Set **A** is **{{250, 3000}, {250, -1}, {3000,-1}}**
- How to enumerate over search space?
 - Using two nested for loops on the input list.

```
for (int i = 0; i < N; ++i){  
    for (int j = i+1; j < N; ++j){
```

Search Space Example 3

Print Binary Strings

Write a recursive function to **print all possible binary strings of length n**, in order (total 2^n binary strings).

Input Format:

One line containing n.

Output Format

The 2^n binary strings in order and separated by a space

Input Sample

3

Output Sample

000 001 010 011 100 101 110 111

- Here, it is required to print the whole search space, all the numbers from 000 to 111
- Set **A** is {000, 001, 010, 011, 100, 101, 110, 111}
- How to enumerate over

2	* 2	* 2
possibilities	possibilities	possibility
(0 or 1)	(0 or 1)	(0 or 1)

Binary Strings = 2^N
strings

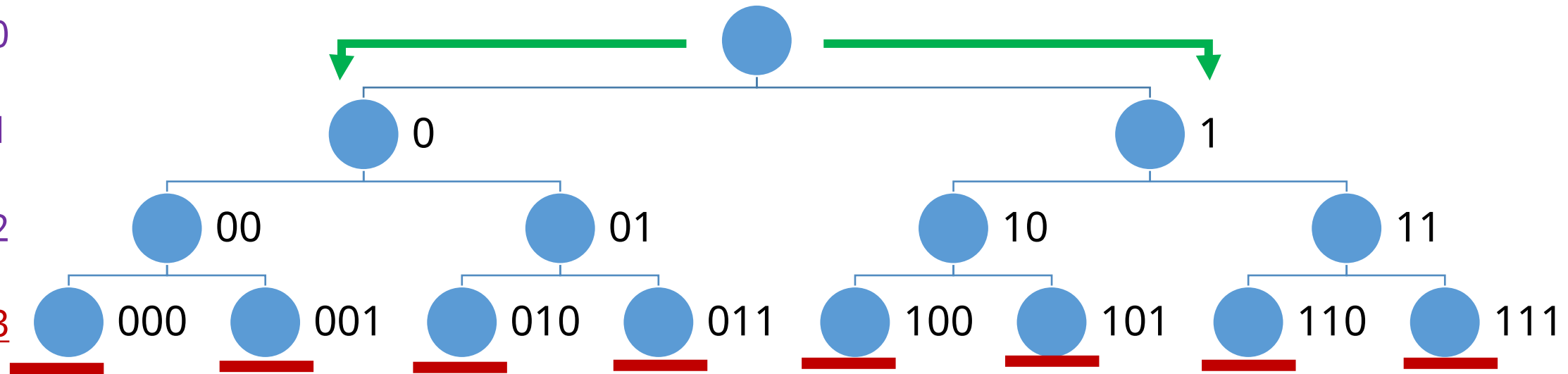
Search Space Example 3

Length = 0

Length = 1

Length = 2

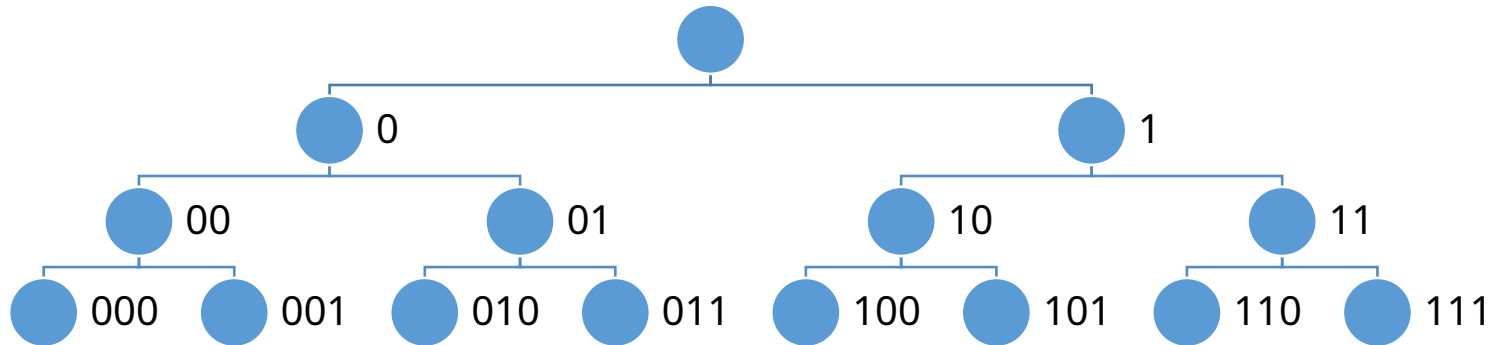
Length = 3



At length == 3, we will print the string.

Search Space Example 3

```
void binary(string s, int n){  
    if(s.length() == n){  
        cout << s << " ";  
    }  
    else  
    {  
        binary(s+"0", n);  
        binary(s+"1", n);  
    }  
}  
  
//call  
binary("", n);
```



Search Space Example 4

Find Binary String

Given m binary strings, find a **binary string of length n containing all m strings as substrings**.

Input Format:

One line containing m and n separated by a space
 m lines containing m binary strings

Output Format

One line containing the binary string

Input Sample

2 4

101

011

Output Sample

1011

- Set **A** is {0000, 0001, 0010, 0011, 0100, 0101, 0110, 0111, 1000, 1001, 1010, **1011**, 1100, 1101, 1110, 1111}
- How to enumerate over search space?
 - Use recursion to generate all binary strings

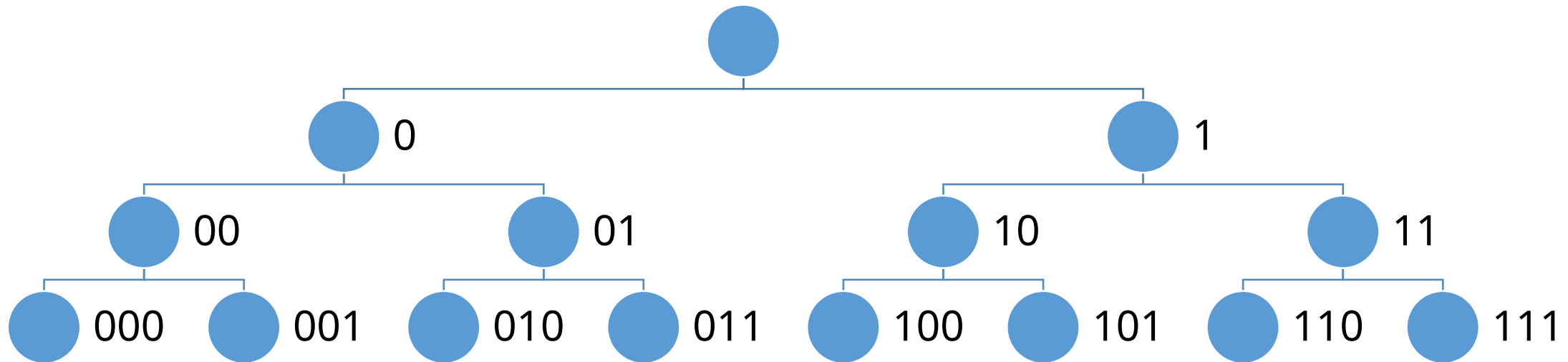
Search Space Example 4

Length = 0

Length = 1

Length = 2

Length = 3



Assuming $n = 3$, then:
at length == 3, we will check if the string matches the condition.

Search Space Example 4

```
string generateString(string s, int n, vector<string> stringList){
```

```
    if(s.length() == n){
```

```
        if(correctString(s, stringList))
```

```
            return s;
```

```
        else
```

```
            return "";
```

```
    }
```

```
    string s1 = generateString(s+"0", n, stringList);
```

```
    string s2 = generateString(s+"1", n, stringList);
```

```
    return s1 == "" ? s2 : s1;
```

```
}
```

```
//call
```

```
cout << generateString("", n, stringList);
```

```
bool correctString(string s, vector<string> stringList)
```

```
{
```

```
    for (int i = 0; i < stringList.size(); i++)
```

```
    {
```

```
        // the following condition means not found in stringList
```

```
        if ( s.find(stringList[i]) == string::npos )
```

```
            return false;
```

```
    }
```

```
    return true;
```

```
}
```

Search Space Example 5

Find Maximum Formed Number

Given n digits find the order of digits that forms the maximum number.

Input Format:

One line containing n

One line containing n digits separated by spaces

Output Format

One line containing the maximum number

Input Sample

3

3 7 5

Output Sample

753

- Set **A** is {375, 357, **753**, 735, 537, 573}}

- How to enumerate over search space?

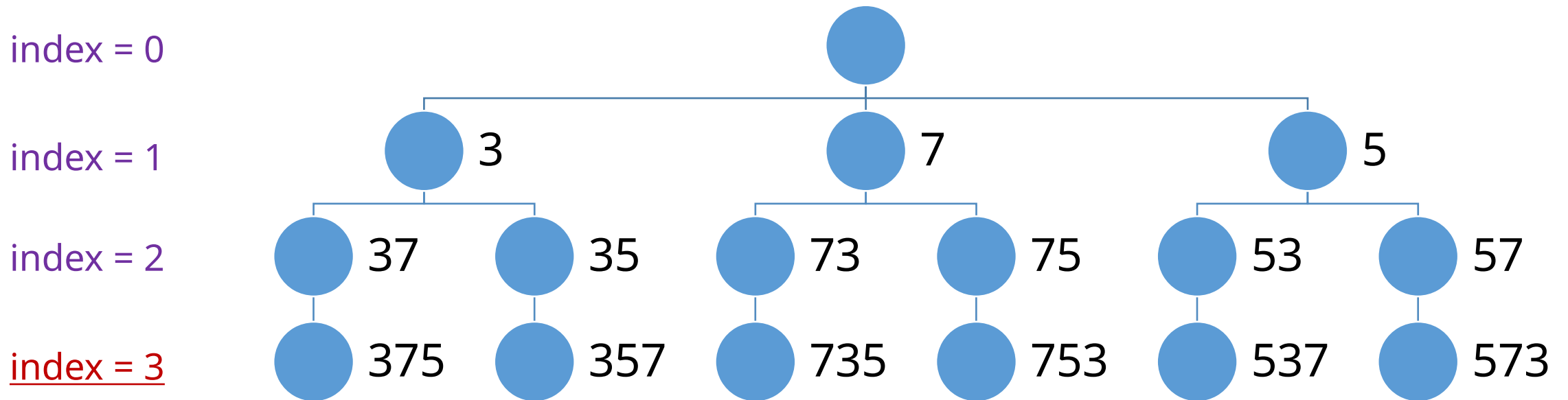
- Use
per

3	7	5
7	5	7
5	3	5
	5	3
	3	7
	7	3

3	* 2	* 1
possibilities	possibilities	possibility

Permutations = N!

Search Space Example 5



- **Base Case:** if number of digits = 3 $\rightarrow$ if (index == n)
- **# Recursive Calls:** which is the number of branches
 - first level: 3 branches, second level: 2, third level: 1
 - Then, we need “visited” or “isused” boolean array/vector

Search Space Example 5

```
void generateNumber(int number, vector<int> digits, vector<bool> isused, int& max, int n, int index){
```

```
    if( index == n){
```

```
        if(number > max)
```

```
            max = number;
```

```
        return;
```

```
    }
```

```
    for(int i = 0; i < n; ++i){
```

```
        if( ! isused[i] ){
```

```
            isused[i] = true;
```

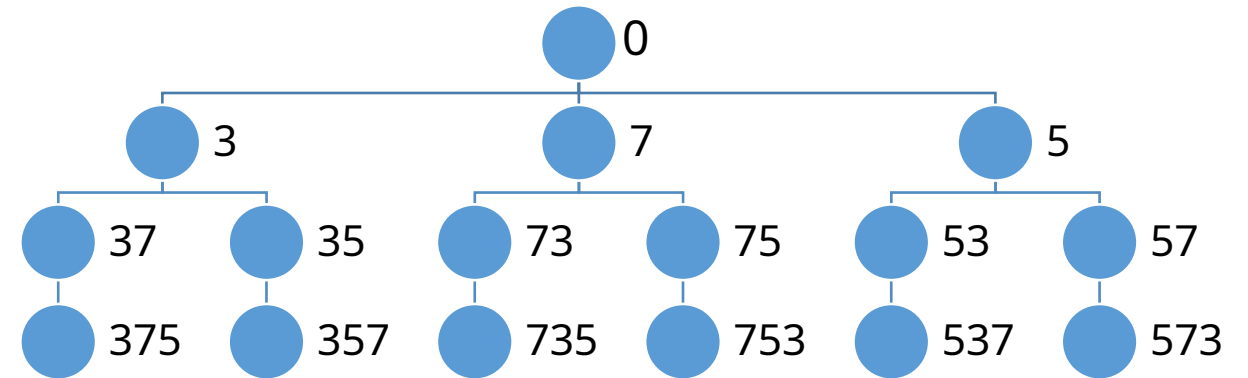
```
            generateNumber(number * 10 + digits[i],
```

```
                digits, isused, max, n, index+1);
```

```
            isused[i] = false;
```

```
        }
```

```
    }
```



```
//call
```

```
int max = INT_MIN;
```

```
vector<bool> isused ( n, false );
```

```
generateNumber(0, digits, isused, max, n, 0);
```